

调试 .NET 中的内存泄漏

项目 • 2023/11/20

本文适用于：  .NET Core 3.1 SDK 及更高版本

当应用引用不再需要执行所需任务的对象时，可能会发生内存泄漏。引用这些对象可以防止垃圾回收器回收所使用的内存。这可能会导致性能下降并引发 [OutOfMemoryException](#) 异常。

本教程演示如何使用 .NET 诊断 CLI 工具分析 .NET 应用中的内存泄漏。如果所在的操作系统是 Windows，则可以[使用 Visual Studio 的内存诊断工具](#)调试内存泄漏。

本教程使用一个有意泄漏内存的示例应用作为练习。还可以分析无意中泄漏内存的应用。

在本教程中，你将：

- ✓ 使用 [dotnet-counters](#) 检查托管内存的使用情况。
- ✓ 生成转储文件。
- ✓ 使用转储文件分析内存使用情况。

先决条件

本教程使用：

- [.NET Core 3.1 SDK](#) 或更高版本。
- [dotnet-counters](#) 检查托管内存的使用情况。
- [dotnet-dump](#) 用于收集和分析转储文件（包括 [SOS 调试扩展](#)）。
- 要诊断的[示例调试目标](#)应用。

本教程假设已安装示例和工具并可供使用。

检查托管内存的使用情况

在开始收集诊断数据以帮助分析本案例的根本原因时，请确保实际看到的是内存泄漏（内存使用增加）。可以使用 [dotnet-counters](#) 工具进行确认。

打开控制台窗口并导航到下载并解压缩[示例调试目标](#)的目录。运行目标：

```
.NET CLI
```

```
dotnet run
```

在单独的控制台中，找到处理 ID：

控制台

dotnet-counters ps

输出应如下所示：

控制台

4807 DiagnosticScena
/home/user/git/samples/core/diagnostics/DiagnosticScenarios/bin/Debug/netcor
eapp3.0/DiagnosticScenarios

现使用 `dotnet-counters` 工具检查托管内存的使用情况。 `--refresh-interval` 指定两次刷新之间的秒数：

控制台

dotnet-counters monitor --refresh-interval 1 -p 4807

实时输出应如下所示：

控制台

Press p to pause, r to resume, q to quit.
Status: Running

[System.Runtime]
of Assemblies Loaded 118
% Time in GC (since last GC) 0
Allocation Rate (Bytes / sec) 37,896
CPU Usage (%) 0
Exceptions / sec 0
GC Heap Size (MB) 4
Gen 0 GC / sec 0
Gen 0 Size (B) 0
Gen 1 GC / sec 0
Gen 1 Size (B) 0
Gen 2 GC / sec 0
Gen 2 Size (B) 0
LOH Size (B) 0
Monitor Lock Contention Count / sec 0
Number of Active Timers 1
ThreadPool Completed Work Items / sec 10
ThreadPool Queue Length 0
ThreadPool Threads Count 1
Working Set (MB) 83

重点介绍此行：

控制台	
GC Heap Size (MB)	4

启动后，可以看到托管堆内存为 4 MB。

现在，转到 URL <https://localhost:5001/api/diagscenario/memleak/20000>。

请注意，内存使用量已增加到 30 MB。

控制台	
GC Heap Size (MB)	30

通过监视内存使用情况，可以确定内存正在增长或泄漏。下一步是收集内存分析的适当数据。

生成内存转储

分析可能的内存泄漏时，需要访问应用的内存堆来分析内存内容。查看对象之间的关系，可以创建理论说明内存未释放的原因。常见的诊断数据源是 Windows 上的内存转储或 Linux 上的等效核心转储。若要生成 .NET 应用程序转储，可使用 `dotnet-dump` 工具。

使用之前启动的[示例调试目标](#)，运行以下命令以生成 Linux 核心转储：

.NET CLI
<code>dotnet-dump collect -p 4807</code>

结果是位于同一文件夹中的核心转储。

控制台
Writing minidump with heap to ./core_20190430_185145 Complete

① 备注

为了随时间推移进行比较，让原始进程在收集第一个转储后继续运行，并采用相同的方式收集第二个转储。然后，你将在一段时间内有两个转储，你可以比较这些转

储，以查看内存使用量的增长位置。

重新启动失败的进程

收集转储后，你应该有足够的信息来诊断失败的进程。如果失败的进程在生产服务器上运行，现在是通过重新启动进程进行短期修正的理想时机。

在本教程中，你已经完成了[示例调试目标](#)，现在可以将其关闭。导航到启动服务器的终端并按 `Ctrl+C`。

分析核心转储

生成核心转储后，请使用 `dotnet-dump` 工具分析转储：

```
.NET CLI
```

```
dotnet-dump analyze core_20190430_185145
```

其中 `core_20190430_185145` 是要分析的核心转储的名称。

❗ 备注

如果你看到报错“找不到 `libdl.so`”，则可能需要安装 `libc6-dev` 包。有关详细信息，请参阅 [Linux 上 .NET 的先决条件](#)。

此时会显示一个提示，可在其中输入 `SOS` 命令。通常，首先要查看的是托管堆的整体状态：

```
控制台
```

```
> dumpheap -stat
```

```
Statistics:
```

	MT	Count	TotalSize	Class Name
...				
00007f6c1eeefba8		576	59904	System.Reflection.RuntimeMethodInfo
00007f6c1dc021c8		1749	95696	System.SByte[]
00000000008c9db0		3847	116080	Free
00007f6c1e784a18		175	128640	System.Char[]
00007f6c1dbf5510		217	133504	System.Object[]
00007f6c1dc014c0		467	416464	System.Byte[]
00007f6c21625038		6	4063376	testwebapi.Controllers.Customer[]
00007f6c20a67498		200000	4800000	testwebapi.Controllers.Customer

```
00007f6c1dc00f90    206770    19494060 System.String
Total 428516 objects
```

可在此处看到大多数对象是 String 或 Customer 对象。

可以使用方法表 (MT) 再次使用 dumpheap 命令来获取所有 String 实例的列表：

控制台

```
> dumpheap -mt 00007f6c1dc00f90

           Address             MT           Size
...
00007f6ad09421f8 00007faddaa50f90           94
...
00007f6ad0965b20 00007f6c1dc00f90           80
00007f6ad0965c10 00007f6c1dc00f90           80
00007f6ad0965d00 00007f6c1dc00f90           80
00007f6ad0965df0 00007f6c1dc00f90           80
00007f6ad0965ee0 00007f6c1dc00f90           80

Statistics:
           MT      Count      TotalSize Class Name
00007f6c1dc00f90    206770    19494060 System.String
Total 206770 objects
```

现在可以对 System.String 实例使用 gcroot 命令，以查看对象的根方式和原因：

控制台

```
> gcroot 00007f6ad09421f8

Thread 3f68:
    00007F6795BB58A0 00007F6C1D7D0745
System.Diagnostics.Tracing.CounterGroup.PollForValues()
[/_/src/System.Private.CoreLib/shared/System/Diagnostics/Tracing/CounterGroup.cs @ 260]
    rbx: (interior)
        -> 00007F6BDFFF038 System.Object[]
        -> 00007F69D0033570 testwebapi.Controllers.Processor
        -> 00007F69D0033588 testwebapi.Controllers.CustomerCache
        -> 00007F69D00335A0
System.Collections.Generic.List`1[[testwebapi.Controllers.Customer,
DiagnosticScenarios]]
        -> 00007F6C000148A0 testwebapi.Controllers.Customer[]
        -> 00007F6AD0942258 testwebapi.Controllers.Customer
        -> 00007F6AD09421F8 System.String

HandleTable:
    00007F6C98BB15F8 (pinned handle)
    -> 00007F6BDFFF038 System.Object[]
```

```
-> 00007F69D0033570 testwebapi.Controllers.Processor
-> 00007F69D0033588 testwebapi.Controllers.CustomerCache
-> 00007F69D00335A0
System.Collections.Generic.List`1[[testwebapi.Controllers.Customer,
DiagnosticScenarios]]
-> 00007F6C000148A0 testwebapi.Controllers.Customer[]
-> 00007F6AD0942258 testwebapi.Controllers.Customer
-> 00007F6AD09421F8 System.String
```

Found 2 roots.

可以看到 String 由 Customer 对象直接保存，并由 CustomerCache 对象间接保存。

可以继续转储对象，以查看大多数 String 对象是否遵循类似的模式。此时，调查会提供足够的信息来确定代码中的根本原因。

可通过此常规过程确定主要内存泄漏源。

清理资源

在本教程中，你已启动一个示例 Web 服务器。此服务器应已关闭，如[重新启动失败的进程](#)部分所述。

还可以删除已创建的转储文件。

请参阅

- 用于列出进程的 [dotnet-trace](#)
- 用于检查托管内存使用情况的 [dotnet-counters](#)
- 用于收集和分析转储文件的 [dotnet-dump](#)
- [dotnet/diagnostics](#)
- [使用 Visual Studio 调试内存泄漏](#)

后续步骤

调试 .NET Core 中的高 CPU